

RAPPORT DE PROJET

DevOps — Application Bancaire Django

El-Mehdi Taoufik | Bilal Chbanat | Mohamed Reda Bouchaiba

Mini-Projet DevOps | 2025–2026

Mai 2026

Django 4.2	Docker	GitHub Actions	SQLite	Gunicorn
------------	--------	----------------	--------	----------

1. Introduction

Ce rapport présente le mini-projet DevOps réalisé dans le cadre du cours. L'objectif est de mettre en pratique les principes DevOps à travers une application web réelle : une plateforme de gestion bancaire développée avec Django.

Le projet couvre l'ensemble du cycle de vie du logiciel : développement, conteneurisation, tests automatisés, intégration continue et déploiement continu (CI/CD).

1.1 Objectifs du projet

- Développer une application bancaire fonctionnelle avec Django
- Conteneuriser l'application avec Docker et Docker Compose
- Mettre en place un pipeline CI/CD avec GitHub Actions
- Automatiser les tests et la publication de l'image Docker
- Documenter les choix techniques et l'architecture

2. Présentation de l'Application

L'application est une plateforme de gestion bancaire complète permettant à un administrateur de gérer les clients et les comptes, et aux clients de consulter leurs soldes et d'effectuer des virements.

2.1 Fonctionnalités

- Authentification (login / logout / inscription)
- Dashboard client : liste des comptes, soldes, transactions récentes
- Dashboard administrateur : vue globale de tous les clients et comptes
- Gestion des comptes : ajout, modification, suppression
- Virements bancaires entre comptes avec validation atomique
- Historique des transactions avec filtre crédit/débit
- Export PDF des relevés de compte (via ReportLab)
- Recherche et filtrage des comptes

2.2 Modèle de données

L'application repose sur trois modèles principaux :

Modèle	Champs principaux	Description
Client	user (OneToOne), phone	Profil étendu de l'utilisateur Django
Account	client, account_number, balance	Compte bancaire lié à un client
Transaction	account, type (CREDIT/DEBIT), amount, date	Opération enregistrée sur un compte

3. Architecture Technique

3.1 Stack technologique

Framework	Django 4.2.16 (Python 3.12)
Base de données	SQLite (persistée via volume Docker)
Serveur WSGI	Gunicorn 23.0.0
Génération PDF	ReportLab 4.2.5
Conteneurisation	Docker (image python:3.12-slim)
Orchestration	Docker Compose v2
CI/CD	GitHub Actions
Registre d'images	GitHub Container Registry (ghcr.io)

3.2 Structure du projet

devops/	
bank/	# Application Django
banking/	# Module principal
models.py	# Client, Account, Transaction
views.py	# Logique métier (12 vues)
forms.py	# AccountForm, TransferForm, etc.
urls.py	# Routage de l'application
templates/	# Templates HTML
static/	# CSS personnalisé
bank_project/	# Configuration Django
settings.py	# Paramètres (env vars)
.github/workflows/	# Pipeline CI/CD
Dockerfile	# Image Docker
docker-compose.yml	# Orchestration
requirements.txt	# Dépendances Python

4. Conteneurisation avec Docker

4.1 Dockerfile

Le Dockerfile utilise une image de base python:3.12-slim pour minimiser la taille de l'image finale. Les bonnes pratiques suivantes ont été appliquées :

- Variables d'environnement PYTHONDONTWRITEBYTECODE et PYTHONUNBUFFERED configurées
- Installation des dépendances avant la copie du code (optimisation du cache Docker)
- Exécution de migrate, collectstatic et démarrage de Gunicorn en une seule commande CMD
- Exposition du port 8000

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY bank ./bank
WORKDIR /app/bank
RUN mkdir -p data

EXPOSE 8000
CMD ["sh", "-c", "python manage.py migrate && \
    python manage.py collectstatic --noinput && \
    gunicorn bank_project.wsgi:application --bind 0.0.0.0:8000"]
```

4.2 Docker Compose

Le fichier docker-compose.yml définit un service unique web avec les caractéristiques suivantes :

- Politique de redémarrage : unless-stopped
- Variables d'environnement externalisées (SECRET_KEY, DEBUG, ALLOWED_HOSTS, SQLITE_PATH)
- Volume nommé sqlite-data pour la persistance de la base de données
- Exposition du port 8000 vers l'hôte

```
services:
  web:
    build: .
    container_name: devops-django-bank
    restart: unless-stopped
    ports:
      - "8000:8000"
    environment:
      SECRET_KEY: ${SECRET_KEY:-change-me}
      DEBUG: ${DEBUG:-True}
      ALLOWED_HOSTS: ${ALLOWED_HOSTS:-localhost,127.0.0.1,0.0.0.0}
      SQLITE_PATH: /app/bank/data/db.sqlite3
    volumes:
      - sqlite-data:/app/bank/data
volumes:
  sqlite-data:
```

4.3 Configuration par variables d'environnement

Le fichier settings.py lit les paramètres sensibles depuis les variables d'environnement, avec des valeurs par défaut pour le développement local. Cette approche respecte le principe 12-factor App et facilite le déploiement en différents environnements (dev, staging, production).

5. Pipeline CI/CD avec GitHub Actions

5.1 Déclencheurs

Le workflow se déclenche sur :

- Push sur les branches main et develop
- Pull Requests ciblant main et develop

5.2 Jobs du pipeline

Le pipeline est composé de deux jobs séquentiels :

Job	Étapes	Condition
build-test	Checkout, setup Python 3.12 pip install, manage.py check manage.py test, docker build	Tous les push et PR
publish-image	Login GHCR, build & push ghcr.io/hasanpiker5543/ devops-django-bank:latest	Push sur main uniquement (après build-test)

5.3 Sécurité du pipeline

Les permissions sont limitées au minimum nécessaire : lecture du contenu du dépôt (contents: read) et écriture sur le registre de packages (packages: write). Le token GITHUB_TOKEN est utilisé automatiquement pour l'authentification au Container Registry, sans nécessiter de secrets supplémentaires.

6. Tests Automatisés

6.1 Structure des tests

Les tests sont organisés dans le fichier banking/tests.py et utilisent le framework de test intégré de Django (TestCase). Deux classes de tests ont été implémentées :

```
class BankingModelTests(TestCase):
    def test_client_and_account_creation(self):
        # Vérifie la création d'un User, Client et Account
        # et leurs relations (str, FK, solde)

class BankingViewTests(TestCase):
    def test_login_page_is_available(self):
        # Vérifie que la page de login répond avec HTTP 200
```

6.2 Couverture des tests

- Test du modèle Client : vérification de __str__, association avec User

- Test du modèle Account : vérification du solde initial, liaison avec Client
- Test de la vue login : statut HTTP 200, disponibilité sans authentification

6.3 Exécution des tests

Les tests s'exécutent localement avec la commande suivante :

```
cd bank
python manage.py test
```

Dans le pipeline CI/CD, ils sont lancés automatiquement à chaque push, garantissant qu'aucune régression n'est introduite.

7. Sécurité

7.1 Points forts

- SECRET_KEY chargée depuis une variable d'environnement (jamais codée en dur en production)
- Authentification obligatoire sur toutes les vues (décorateur @login_required)
- Vérification des droits sur chaque vue admin (is_superuser)
- Virements atomiques (db_transaction.atomic()) : pas de perte de données en cas d'erreur
- Protection CSRF activée (CsrfViewMiddleware)
- Validation des formulaires côté serveur (solde insuffisant, compte inexistant, virement vers soi-même)

7.2 Points d'amélioration

- La vue delete_account ne vérifie pas is_superuser avant la suppression POST
- Le mot de passe par défaut ("12345678") lors de la création admin est un risque en production
- Passer à PostgreSQL pour un déploiement en production
- Ajouter HTTPS (reverse proxy Nginx + certificat TLS)
- Activer les en-têtes de sécurité Django (SECURE_SSL_REDIRECT, HSTS, etc.)

8. Guide de Déploiement

8.1 Lancement local (sans Docker)

```
pip install -r requirements.txt
cd bank
python manage.py migrate
python manage.py runserver
```

```
# Accès : http://127.0.0.1:8000
```

8.2 Lancement avec Docker Compose

```
# Copier et configurer les variables d'environnement
cp .env.example .env

# Construire et lancer
docker compose up --build
# Accès : http://localhost:8000
```

8.3 Utilisation de l'image publiée

```
docker pull ghcr.io/hasanpiker5543/devops-django-bank:latest
docker run -p 8000:8000 \
  -e SECRET_KEY=votre-cle-secrete \
  -e DEBUG=False \
  ghcr.io/hasanpiker5543/devops-django-bank:latest
```

9. Conclusion

Ce projet a permis d'appliquer concrètement les pratiques DevOps sur une application réelle. L'ensemble du cycle — développement, conteneurisation, tests et livraison continue — est automatisé et reproductible.

Les points clés réalisés :

- Application Django fonctionnelle avec authentification, gestion bancaire complète et export PDF
- Image Docker optimisée basée sur python:3.12-slim
- Orchestration simple avec Docker Compose et persistance des données via volume
- Pipeline CI/CD GitHub Actions avec tests automatisés, build Docker et publication sur GHCR
- Configuration externalisée respectant le principe 12-factor App

Les prochaines évolutions pourraient inclure : migration vers PostgreSQL, mise en place d'un reverse proxy Nginx, ajout de tests de couverture plus exhaustifs, et monitoring applicatif.